PROGINEER · WEEKLY TASK · REPORT

Advanced Filtering, Caching & Performance

Showroom Management API · Izzaldeen · November 2026 · ASP.NET Core (.NET 10) · EF Core 10 · SQL Server · IMemoryCache
Repository: github.com/aldinizz2004-png/ShowroomManagement-Testing

100,000	71 → 9.6 ms	< 1 ms	292	4 PRs
vehicles in the performance database	available vehicles by price, index added	every list request served from cache	tests passing (164 unit + 128 integration)	filtering, caching, indexes, review (#8 to #11)

1 Filtering, sorting and pagination

Parameter	Behaviour
`brand`	Chery, OMODA, JAECOO
`model`	text contained in the model (escaped LIKE)
`year`	exact manufacturing year
`minPrice` , `maxPrice`	inclusive range in NIS
`color` , `status`	exact match (customers always `Available`)
`sortBy` , `sortOrder`	price / model / year, asc / desc, ties by id
`page` , `pageSize`	from 1; size 1 to 50, default 10

Validation
Page < 1, page size > 50, negative or reversed prices, year out of range, unknown sort field or order → **400** with the field name. Wrong types or unknown enums are rejected by model binding.

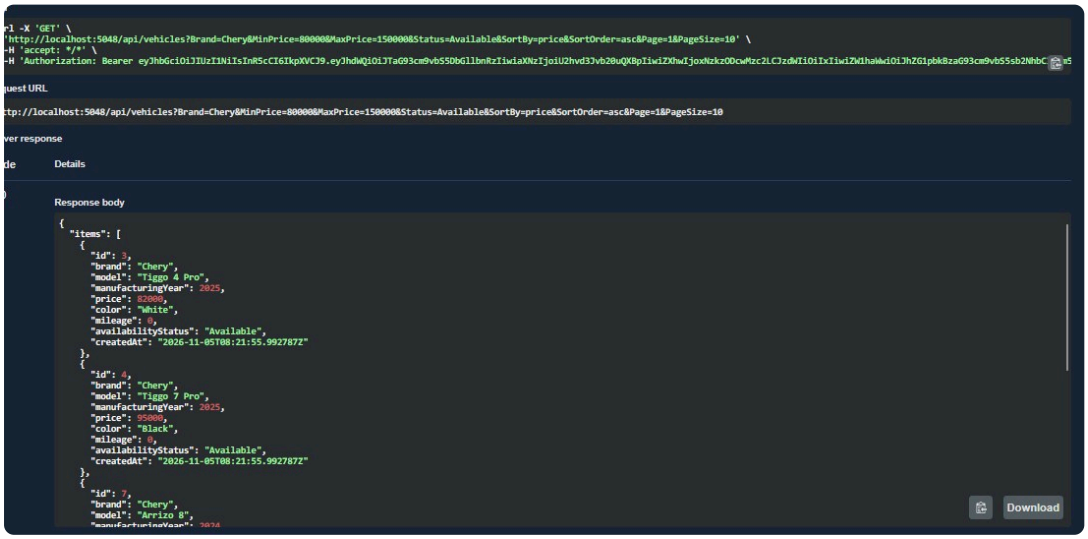
```
var query =
_db.Vehicles.AsNoTracking().TagWith("Vehicle
search");
if (search.Brand is { } brand)
    query = query.Where(v => v.Brand == brand);
if (search.MinPrice is { } min)
    query = query.Where(v => v.Price >= min);
...

var total = await query.CountAsync(ct);
var items = await query.OrderBy(v =>
v.Price).ThenBy(v => v.Id)
    .Skip((page - 1) * pageSize).Take(pageSize)
    .ToListAsync(ct);
```

```
{ "items": [...], "page": 1, "pageSize": 10,
"totalCount": 23,
  "totalPages": 3, "hasPreviousPage": false,
"hasNextPage": true }
```

Nothing is filtered in memory: `ToListAsync` is the only materialization and it returns one page. The generated SQL, checked by a test with `ToQueryString()` :

```
-- Vehicle search
SELECT [v].[Id], [v].[AvailabilityStatus], [v].[Brand], ... FROM [Vehicles] AS [v]
WHERE [v].[Brand] = @brand AND [v].[ManufacturingYear] = @year AND [v].[Price] >= @minPrice AND [v].[Price] <=
@maxPrice
ORDER BY [v].[Price], [v].[Id]
OFFSET @p ROWS FETCH NEXT @p5 ROWS ONLY
```



Swagger: the specification example returns Chery vehicles in the price range, cheapest first (82,000 then 95,000 ...).

2 Caching with IMemoryCache

Entry	Key	Sliding	Absolute
Vehicle list	`vehicles:list:{search}`	1 min	5 min
Vehicle details	`vehicles:detail:{id}`	2 min	10 min

The list key is the full `VehicleSearch` record (filters, sort, page, size), so each combination is cached separately, and a customer's search always contains `Status = Available`. Expirations are configurable in `appsettings.json`; the cache can be switched off for measurements. Size limit: 1,024 entries.

Sliding vs absolute
Sliding removes entries nobody asks for; absolute guarantees that even a popular entry is rebuilt after a fixed time.

Write	Invalidated
POST vehicle	all lists
PUT vehicle	that vehicle + all lists
PATCH status	that vehicle + all lists
DELETE vehicle	that vehicle + all lists
Reservation create / cancel / complete	that vehicle + all lists (status changes)

How freshness is guaranteed
All list entries share one `CancellationChangeToken`; cancelling it evicts every list at once. A version counter is read before each database load: if an invalidation happens during the load, the result is returned but **not cached**, so stale data cannot be stored after a change.

Request
GET /api/vehicles?...

→

VehicleCache
key found → return (hit)
not found → load (miss)

→ miss

Repository
COUNT + one page from SQL Server

→

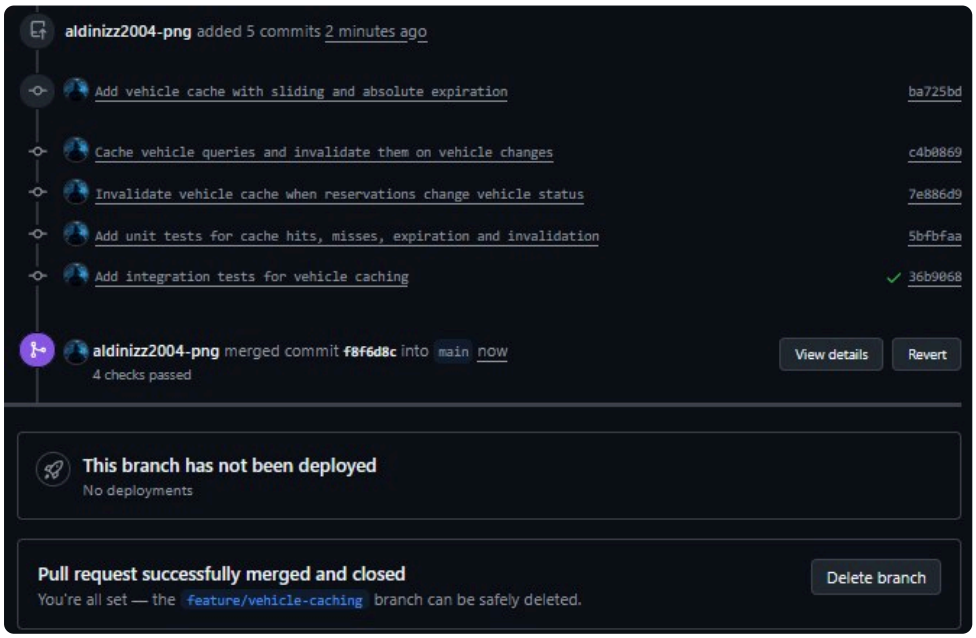
Store
with sliding + absolute expiration and the list token

3 Tests added

Unit (164 total, +53)
Query validation for every invalid parameter, sort mapping, pagination metadata, all criteria passed to the repository. `VehicleCache` with a real `MemoryCache`: miss, hit, separate keys, missing vehicle not cached, **sliding and absolute expiration**, invalidation of one vehicle and of all lists, disabled mode, invalidation during load. Service and reservations: writes clear the right entries, failed writes clear nothing.

Integration (128 total, +49)
Every filter alone and combined, the specification URL, sorting both ways for price / model / year, page metadata, every vehicle exactly once across pages, empty results, wildcard as text, 400 cases. **Cache hit proof:** a price changed directly in the database stays hidden while cached; after POST / PUT / PATCH / DELETE / reservation the next read is fresh. SQL Server SQL shape and EF index metadata.

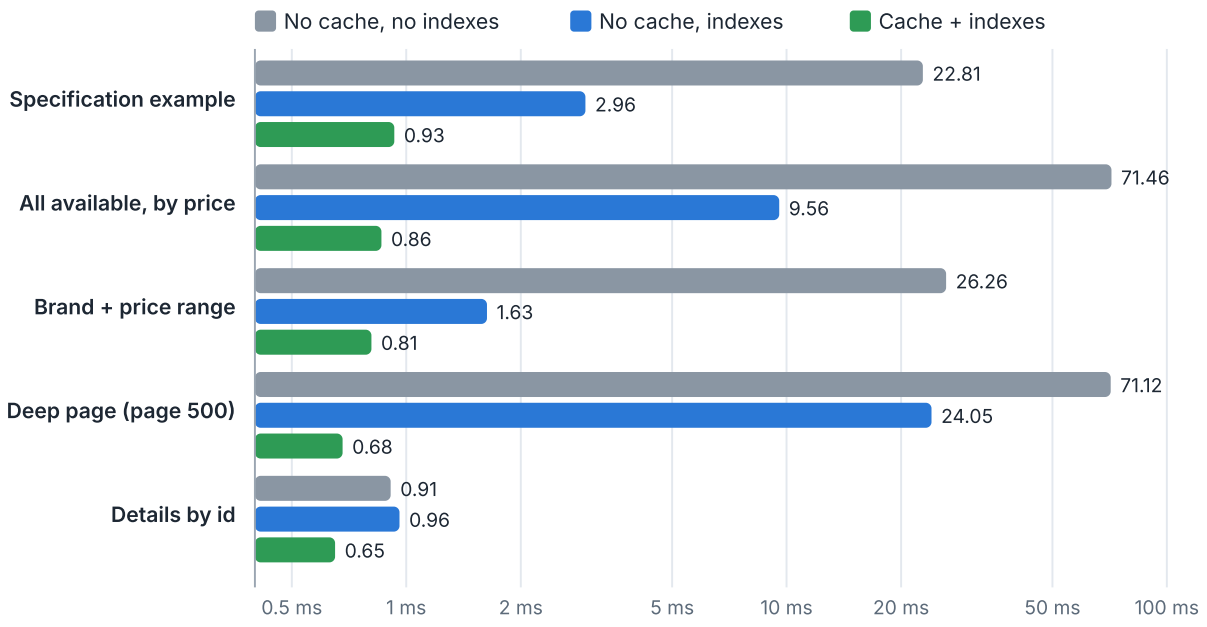
Coverage (unit + integration): **99.3 %** lines, **94.1 %** branches. All tests run in GitHub Actions on every push and pull request.



PR #9 (caching): five focused commits, all checks passed, merged into `main`.

4 Performance review

Setup: 100,000 vehicles (3 brands, 9 years, 70 % available) in a separate database `ShowroomPerf`, Release build, SQL logging off, customer token, 10 warm-up + 200 sequential requests per scenario (`perf/measure.ps1`). Three configurations: indexes dropped and cache off; indexes on and cache off; both on.



Average end-to-end latency per request (log scale, milliseconds). Raw data in `perf/results.csv`.

Scenario	No cache, no indexes avg / p95	No cache, indexes avg / p95	Cache + indexes avg / p95
Specification example (Chery, 2025, 80k to 150k, by price)	22.81 / 29.39	2.96 / 5.16	**0.93 / 1.47**
All available, sorted by price, page 1	71.46 / 94.86	9.56 / 14.71	**0.86 / 1.37**
OMODA, 100k to 120k, by price	26.26 / 28.84	1.63 / 2.83	**0.81 / 1.12**
Deep page (page 500 of available by price)	71.12 / 78.51	24.05 / 28.29	**0.68 / 0.91**
Vehicle details by id	0.91 / 1.67	0.96 / 1.71	**0.65 / 0.75**

What the numbers show

Indexes make uncached lists **3x to 16x** faster. The cache brings every repeated list request under **1 ms**, 25x to 100x faster than the unindexed database. Details by id are already a primary-key seek, so the cache only saves the round trip there.

Remaining costs

The deep page still takes 24 ms uncached, because `OFFSET 9980` walks past 9,980 index rows (keyset pagination would fix it). "All available" is dominated by `COUNT(*)` over 70,000 rows. Both are cached after the first request.

5 Database indexes and SQL audit

Index	Serves
`(Brand, AvailabilityStatus, Price) INCLUDE (ManufacturingYear)`	brand + status + price range, sorted by price; year checked inside the index
`(AvailabilityStatus, Price)`	all available vehicles by price (every customer list without brand)

Column order: columns compared with `=` first, the range / sort column last. SQL Server seeks straight to *Chery, Available, 80,000* and reads rows already ordered by price, so there is no Sort and `TOP 10` stops after 10 rows.

Removed: `(Brand, Model)`, which `LIKE '%text%'` cannot use, and `(AvailabilityStatus)`, now the prefix of the new index.

Logical reads	No index	v1	Final
Q1 page (spec example, available)	1,519	281	**39**
Q2 count for Q1	1,519	1,519	**76**
Q3 all available by price	1,519	**71**	**71**
Q4 OMODA, 100k to 120k	1,519	**71**	**71**

Audit finding

In v1 the count query still scanned the whole table: the year was not in the index, and thousands of key lookups looked worse to SQL Server than a scan. Adding `ManufacturingYear` as an included column (migration `IncludeYearInBrandIndex`) cut it to 76 reads, and the uncached spec example from 17.3 ms to 3.0 ms.

```
-- query plan without indexes
Sort(TOP 10, ORDER BY Price, Id)  <-  Clustered Index Scan PK_Vehicles   (reads all 100,000 rows, then sorts)
-- query plan with indexes
Top(10)  <-  Index Seek IX_Vehicles_Brand_Status_Price (Brand = 'Chery', Status = 'Available', Price 80k..150k)
ORDERED FORWARD
         +  Key Lookup PK_Vehicles   (only for the 10 returned rows)
```

Trade-off	Measured / reason	Decision
Storage	indexes 5.4 MB + 3.8 MB beside an 11.9 MB table	acceptable for the read gain
Slower writes	bulk insert of 50,000 rows: about 0.6 to 0.9 s with indexes vs 0.3 s without	vehicles are written rarely, read constantly
Not covering	returned rows need a key lookup	only 10 to 50 rows per page; covering would duplicate the table
No index on color, year alone, model	low selectivity (6 colors, 9 years) or leading-wildcard LIKE	applied as residual filters after the seek
Per-instance cache	each API instance has its own memory	fine for one instance; Redis or messaging for several

SQL visibility: search queries are tagged `-- Vehicle search`, SQL commands are logged in Development, and `perf/query-audit.sql` / `query-plans.sql` reproduce the reads and plans above. All queries are parameterized, so plans are reused, and all reads use `AsNoTracking`.

6 Git workflow and deliverables

PR	Branch and content
#8	`feature/vehicle-filtering`: search criteria, validation, database filtering / sorting / paging, unit and integration tests (5 commits)
#9	`feature/vehicle-caching`: cache, service caching and invalidation, reservation invalidation, tests (5 commits)
#10	`feature/vehicle-indexes`: query tagging and SQL logging, composite indexes migration, SQL / index tests (3 commits)
#11	`feature/performance-review`: index refinement after the audit, perf scripts and results, PERFORMANCE.md, README

Details: README.md (parameters, caching, indexes) · PERFORMANCE.md (full results and justification) · TEST_COVERAGE.md · perf/ (seed, measure, audit scripts).